

CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: SISTEMAS DISTRIBUÍDOS

GABRIEL VIANA DA COSTA
MARLON ANDRÉ PEREIRA ALMEIDA
PEDRO VICTOR SOARES DA SILVA ARAÚJO
RANDERSON SOUSA DE SÁ NUNES

LEITOSYNC: SISTEMA DISTRIBUÍDO PARA GERENCIAMENTO DE LEITOS E
RECURSOS HOSPITALARES CRÍTICOS COM JSON-RPC

GABRIEL VIANA DA COSTA
MARLON ANDRÉ PEREIRA ALMEIDA
PEDRO VICTOR SOARES DA SILVA ARAÚJO
RANDERSON SOUSA DE SÁ NUNES

**LEITOSYNC: SISTEMA DISTRIBUÍDO PARA GERENCIAMENTO DE LEITOS E
RECURSOS HOSPITALARES CRÍTICOS COM JSON-RPC**

Relatório prático apresentado à disciplina de Sistemas Distribuídos, como requisito avaliativo parcial, com foco na aplicação do problema clássico dos leitores e escritores em um sistema distribuído baseado em chamadas remotas de procedimento.

Professor: François Fernandes Ribeiro Barbosa.

RESUMO

Este relatório apresenta o LeitoSync, um sistema distribuído para gerenciamento de leitos e recursos hospitalares críticos. O projeto foi desenvolvido para demonstrar, em um cenário aplicado, o problema clássico dos leitores e escritores. No sistema, operações de consulta de disponibilidade representam leitores, enquanto operações de reserva, ocupação, liberação, bloqueio e manutenção representam escritores. A versão final da solução utiliza JSON-RPC 2.0 como mecanismo principal de comunicação entre cliente e servidor, concentrando as operações no endpoint único /rpc. A consistência das alterações é garantida por transações no PostgreSQL e bloqueio pessimista por linha, por meio de `SELECT FOR UPDATE`, evitando reservas duplicadas de um mesmo recurso. A solução também inclui interface web, auditoria, atualização em tempo real e testes automatizados de concorrência. Os resultados demonstram que múltiplas leituras simultâneas são atendidas corretamente, enquanto múltiplas escritas concorrentes sobre o mesmo recurso resultam em apenas uma reserva confirmada.

Palavras-chave: Sistemas Distribuídos. Leitores e Escritores. RPC. JSON-RPC. Controle de Concorrência. PostgreSQL.

Sumário

1	INTRODUÇÃO	4
2	OBJETIVOS	4
3	FUNDAMENTAÇÃO TEÓRICA	5
3.1	Problema dos leitores e escritores	5
3.2	Remote Procedure Call	6
4	METODOLOGIA E DESENVOLVIMENTO	6
4.1	Arquitetura geral da solução	6
4.2	Uso de RPC no LeitoSync	7
4.3	Modelagem do domínio	9
4.4	Operações de leitura	9
4.5	Operações de escrita	10
5	SOLUÇÃO DO PROBLEMA DOS LEITORES E ESCRITORES	10
5.1	Controle transacional com bloqueio pessimista	10
5.2	Tratamento de conflitos	12
5.3	Auditoria	12
5.4	Atualização em tempo real	12
6	RESULTADOS E DISCUSSÃO	13
6.1	Simulador leitores-escretores	13
6.2	Testes automatizados	13
7	EXECUÇÃO DO PROJETO	14
8	SIMULAÇÃO SIMPLIFICADA EM PYTHON	15
9	LIMITAÇÕES	15
10	TRABALHOS FUTUROS	16

11 CONSIDERAÇÕES FINAIS**16****12 REFERÊNCIAS****17**

1 INTRODUÇÃO

O LeitoSync é um sistema distribuído voltado ao gerenciamento de leitos e recursos hospitalares críticos, como leitos de UTI, leitos clínicos, salas cirúrgicas, respiradores e equipamentos médicos essenciais.

Em uma rede de saúde moderna, a gestão desses recursos envolve múltiplas unidades hospitalares, operadores, médicos e centrais de regulação consultando e alterando informações compartilhadas de maneira concorrente. Esse cenário exige mecanismos adequados para evitar inconsistências, especialmente quando diferentes usuários tentam modificar o mesmo recurso ao mesmo tempo.

O objetivo principal deste projeto não é apenas construir um sistema de operações básicas de cadastro, consulta e alteração. O projeto foi concebido para demonstrar, na prática e em um cenário distribuído realista, a aplicação do problema clássico dos leitores e escritores. A solução mostra como uma arquitetura baseada em RPC, banco de dados transacional e bloqueio pessimista pode evitar falhas como a reserva duplicada de um leito crítico.

2 OBJETIVOS

O objetivo geral do projeto é implementar um sistema distribuído para gerenciamento de leitos e recursos hospitalares críticos, demonstrando a aplicação prática do problema dos leitores e escritores por meio de chamadas remotas de procedimento.

Os objetivos específicos são:

- permitir a consulta de hospitais e de seus respectivos recursos hospitalares;
- permitir operações de reserva, ocupação, liberação, bloqueio e manutenção de recursos;
- evitar a reserva duplicada de um mesmo recurso por centrais distintas operando simultaneamente;
- permitir múltiplas leituras simultâneas sem bloqueio exclusivo desnecessário;
- controlar escritas concorrentes sobre o mesmo recurso, preservando a consistência dos dados;
- registrar uma trilha de auditoria das alterações realizadas;
- demonstrar o comportamento concorrente por meio de testes automatizados e de um simulador interativo;

- reformular a comunicação principal do sistema para JSON-RPC 2.0, atendendo ao requisito de uso de RPC em Sistemas Distribuídos.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Problema dos leitores e escritores

O problema dos leitores e escritores ocorre quando múltiplos processos, threads ou clientes acessam um recurso compartilhado. Nesse modelo, vários leitores podem acessar o recurso simultaneamente, pois a leitura não altera o estado do sistema. Porém, escritores precisam de acesso exclusivo, uma vez que a escrita modifica o estado compartilhado e pode causar inconsistências se ocorrer ao mesmo tempo que outra operação de escrita.

Os principais elementos desse problema são:

- **leitores:** entidades que consultam o estado de um recurso;
- **escritores:** entidades que modificam o estado de um recurso;
- **recurso compartilhado:** dado ou estrutura acessada concorrentemente;
- **região crítica:** trecho de código ou transação em que ocorre a alteração do recurso;
- **condição de corrida:** situação em que o resultado depende da ordem de execução das operações concorrentes;
- **exclusão mútua:** garantia de que somente um escritor acesse a região crítica por vez;
- **consistência dos dados:** preservação de um estado correto após operações concorrentes.

No contexto do LeitoSync, os leitores são os métodos RPC de consulta, como `recursos.listar`, `recursos.obter`, `dashboard.resumo` e `auditoria.listar`. Os escritores são os métodos RPC que alteram o estado de um recurso, como `recursos.reservar`, `recursos.liberar`, `recursos.ocupar`, `recursos.bloquear` e `recursos.manutencao`.

Problema clássico	LeitoSync
Leitores	Consultas de hospitais, recursos, dashboard, auditoria e disponibilidade
Escritores	Reservar, ocupar, liberar, bloquear ou colocar recurso em manutenção

Problema clássico	LeitoSync
Recurso compartilhado	Estado dos leitos e recursos hospitalares
Região crítica	Transação de alteração do status e versionamento de um recurso
Risco	Reserva duplicada, estado inconsistente ou perda de atualização
Solução	Transações no PostgreSQL com bloqueio pessimista por linha

3.2 Remote Procedure Call

RPC, ou Remote Procedure Call, é um modelo de comunicação em que um cliente solicita a execução de um procedimento localizado em outro processo ou servidor. A chamada remota é abstraída como uma chamada de função, embora envolva comunicação em rede, serialização de parâmetros, processamento remoto e retorno de resposta.

No LeitoSync, a versão final utiliza JSON-RPC 2.0 como mecanismo principal de comunicação. O cliente envia uma mensagem para o endpoint `/rpc`, informando no campo `method` qual procedimento remoto deve ser executado. O backend recebe a chamada, despacha para a função correspondente e retorna o resultado também no formato JSON-RPC.

O uso de HTTP nesse caso é apenas o transporte da mensagem. A arquitetura não é orientada a múltiplas rotas REST para as operações principais; ela é orientada a procedimentos remotos, como `recursos.listar` e `recursos.reservar`.

4 METODOLOGIA E DESENVOLVIMENTO

4.1 Arquitetura geral da solução

O LeitoSync adota uma arquitetura em camadas, empacotada em containers para facilitar a execução e a reprodutibilidade. As principais tecnologias utilizadas foram:

- **Backend:** Python, FastAPI, SQLAlchemy assíncrono, PostgreSQL, Alembic, Pytest e Web-Socket;

- **Frontend:** React, TypeScript, Vite e Tailwind CSS;
- **Infraestrutura:** Docker e Docker Compose;
- **Comunicação principal:** JSON-RPC 2.0 sobre HTTP, por meio do endpoint `/rpc`.

As responsabilidades foram distribuídas da seguinte forma:

- **Frontend:** interface visual, painel, listagem de recursos, detalhes de recurso, tela de auditoria e simulador de leitores e escritores;
- **Backend:** servidor RPC, regras de negócio, controle de concorrência, integração com banco de dados e emissão de notificações WebSocket;
- **Banco de dados:** persistência, integridade relacional, transações e bloqueio pessimista por linha;
- **Testes:** validação da comunicação RPC, das operações de leitura, das operações de escrita e dos cenários concorrentes.

4.2 Uso de RPC no LeitoSync

O LeitoSync foi reformulado para utilizar JSON-RPC 2.0 como mecanismo principal de comunicação entre cliente e servidor. Em vez de expor várias rotas REST para cada operação, o sistema concentra as chamadas no endpoint `POST /rpc`. Cada requisição informa o procedimento remoto a ser executado no campo `method`.

A Tabela a seguir resume a diferença entre a versão REST anterior e a versão RPC atual.

Aspecto	Versão REST anterior	Versão RPC atual
Entrada principal	Várias rotas HTTP	Endpoint único <code>/rpc</code>
Listagem de recursos	<code>GET /resources</code>	<code>recursos.listar</code>
Reserva de recurso	<code>POST /resources/{id}/reserve</code>	<code>recursos.reservar</code>
Simulação	Endpoints REST	Métodos <code>simulacao.*</code>
Comunicação	Orientada a recursos	Orientada a procedimentos
Consistência	PostgreSQL + lock	PostgreSQL + lock

Exemplo de chamada RPC para reserva de recurso:

```
{
  "jsonrpc": "2.0",
  "method": "recursos.reservar",
  "params": {
    "resource_id": 1,
    "requester_name": "Central de Regulação",
    "patient_code": "PAC-001",
    "priority": "emergency",
    "reason": "Paciente crítico"
  },
  "id": 2
}
```

Exemplo de resposta de sucesso:

```
{
  "jsonrpc": "2.0",
  "result": {
    "success": true,
    "message": "Recurso reservado com sucesso."
  },
  "id": 2
}
```

Exemplo de conflito:

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": 409,
    "message": "0 recurso não está mais disponível para reserva."
  },
  "id": 2
}
```

4.3 Modelagem do domínio

O domínio foi modelado de forma enxuta para validar o comportamento distribuído e concorrente. As principais entidades são:

- **User:** identifica usuários ou responsáveis por operações;
- **Hospital:** representa unidades de saúde distribuídas;
- **ResourceType:** define categorias de recursos, como leito de UTI ou respirador;
- **Resource:** representa o recurso compartilhado principal, como um leito ou equipamento;
- **Reservation:** registra reservas realizadas sobre recursos;
- **AuditLog:** registra alterações de estado para rastreabilidade;
- **ConcurrencyEvent:** armazena eventos associados às simulações de concorrência.

A entidade central é Resource, pois representa o recurso compartilhado protegido pelo sistema. Cada recurso possui hospital associado, tipo de recurso, código, status, versão e datas de criação e atualização.

Os principais estados do recurso são:

- **available:** disponível para reserva;
- **reserved:** reservado;
- **occupied:** ocupado;
- **blocked:** bloqueado;
- **maintenance:** em manutenção.

Os valores internos são mantidos em inglês por estabilidade técnica do banco, dos testes e dos métodos RPC, enquanto a interface apresenta os rótulos em português.

4.4 Operações de leitura

As operações de leitura são métodos RPC que consultam o estado do sistema sem modificá-lo. Entre elas estão:

- `recursos.listar;`
- `recursos.obter;`
- `recursos.disponibilidade;`
- `hospitais.listar;`

- `tipos_recursos.listar`;
- `dashboard.resumo`;
- `auditoria.listar`.

Esses métodos representam os leitores no problema clássico. Múltiplos clientes podem consultar simultaneamente a disponibilidade dos recursos hospitalares sem alterar o estado compartilhado.

4.5 Operações de escrita

As operações de escrita são métodos RPC que alteram o estado de um recurso. Entre elas estão:

- `recursos.reservar`;
- `recursos.liberar`;
- `recursos.ocupar`;
- `recursos.bloquear`;
- `recursos.manutencao`.

Essas operações representam os escritores, pois modificam o estado compartilhado dos leitos e equipamentos. Por esse motivo, precisam ser coordenadas para evitar inconsistências. Sem controle de concorrência, duas centrais poderiam reservar o mesmo leito simultaneamente, causando reserva duplicada e possível encaminhamento incorreto de pacientes.

5 SOLUÇÃO DO PROBLEMA DOS LEITORES E ESCRITORES

5.1 Controle transacional com bloqueio pessimista

A solução de concorrência do LeitoSync está concentrada na camada de repositório, especialmente na operação de reserva. O sistema utiliza transações no PostgreSQL e bloqueio pessimista por linha, equivalente a `SELECT FOR UPDATE`, acionado pelo SQLAlchemy por meio de `with_for_update()`.

O fluxo de reserva é o seguinte:

1. o cliente envia uma chamada JSON-RPC para `/rpc`, usando o método `recursos.reservar`;
2. o backend identifica o método e executa a função correspondente;

3. uma transação é aberta no banco;
4. o recurso é buscado com bloqueio pessimista por linha;
5. o sistema verifica se o status ainda é `available`;
6. se o recurso estiver disponível, o status é alterado para `reserved`;
7. a versão do recurso é incrementada;
8. uma reserva é criada;
9. um registro de auditoria é criado;
10. a transação é confirmada;
11. os clientes conectados podem ser notificados por WebSocket;
12. se outro escritor tentar reservar o mesmo recurso, ele encontra o status atualizado e recebe erro de conflito com código 409.

Pseudocódigo da operação:

BEGIN TRANSACTION

SELECT recurso

FROM resources

WHERE id = resource_id

FOR UPDATE

IF recurso.status != available:

ROLLBACK

RETURN 409 Conflict

UPDATE resources

SET status = reserved,

 version = version + 1

INSERT INTO reservations (...)

INSERT INTO audit_logs (...)

COMMIT

Essa estratégia evita a condição de corrida conhecida como reserva duplicada. Mesmo que várias chamadas RPC de escrita cheguem quase ao mesmo tempo, apenas uma transação consegue confirmar a reserva do recurso.

5.2 Tratamento de conflitos

Quando um recurso não está mais disponível para reserva, o sistema retorna um erro JSON-RPC com código 409. Esse código representa conflito de estado: a requisição é sintaticamente válida, mas não pode ser concluída porque o estado do recurso mudou.

Um exemplo prático ocorre quando duas centrais visualizam o mesmo leito como disponível e tentam reservá-lo quase simultaneamente. A primeira transação adquire o bloqueio, altera o status para reservado e confirma a operação. A segunda transação, ao verificar o estado atualizado, identifica que o recurso já não está disponível e recebe conflito. Assim, o sistema impede que duas reservas sejam confirmadas para o mesmo leito.

5.3 Auditoria

A entidade `AuditLog` registra as alterações de estado dos recursos. Toda escrita relevante cria um registro contendo o recurso alterado, a ação realizada, o status anterior, o novo status, o responsável e a data da operação.

A auditoria é criada dentro da mesma transação que altera o recurso. Dessa forma, se a transação falhar e sofrer rollback, o log também não é persistido. Isso evita registros órfãos e mantém a rastreabilidade das alterações consistente.

5.4 Atualização em tempo real

O `LeitoSync` também utiliza `WebSocket` para atualização da interface. Quando uma operação de escrita é confirmada, o backend pode emitir um evento para os clientes conectados, informando que o estado de um recurso foi alterado.

O `WebSocket` não é o mecanismo de consistência do sistema. A consistência é garantida pelo banco de dados, pelas transações e pelo bloqueio pessimista. O `WebSocket` atua apenas como mecanismo de notificação para melhorar a experiência da interface.

6 RESULTADOS E DISCUSSÃO

6.1 Simulador leitores-escretores

O frontend inclui uma tela de simulador para demonstrar visualmente o problema dos leitores e escritores. O simulador permite disparar múltiplas chamadas de leitura e múltiplas chamadas de escrita sobre o mesmo recurso.

Os resultados esperados são:

- múltiplos leitores consultam recursos simultaneamente;
- múltiplos escritores tentam reservar o mesmo recurso;
- apenas uma escrita é confirmada;
- as demais escritas retornam conflito;
- o estado final do recurso permanece consistente.

Essa funcionalidade torna visível a aplicação prática do problema clássico em um cenário de gestão hospitalar.

6.2 Testes automatizados

A suíte de testes foi implementada com Pytest e executada dentro do container do backend. Após a refatoração para RPC, os testes foram reescritos para chamar o endpoint `/rpc` com payloads JSON-RPC.

Foram validados oito testes principais:

1. `test_rpc_get_resources`: valida a listagem de recursos por RPC;
2. `test_rpc_reserve_available_resource`: valida a reserva de um recurso disponível;
3. `test_rpc_reserve_already_reserved_resource`: valida o conflito ao tentar reservar um recurso já reservado;
4. `test_rpc_concurrent_reservations`: executa múltiplas reservas concorrentes e confirma que apenas uma é aceita;
5. `test_rpc_concurrent_readers`: executa múltiplas leituras simultâneas;
6. `test_rpc_readers_during_writer_attempt`: valida leitores durante tentativa de escrita;
7. `test_rpc_reservation_creates_single_active_reservation`: confirma que apenas uma reserva ativa é criada;

8. `test_rpc_write_creates_audit_log`: confirma que uma escrita gera log de auditoria.

O resultado final da execução foi:

8 passed, 1 warning

O aviso apresentado está relacionado à configuração de classes no Pydantic e não afeta a execução nem a correteza da solução.

7 EXECUÇÃO DO PROJETO

O projeto é executado por Docker Compose. Na raiz do projeto, utiliza-se:

```
docker compose up --build
```

Com os containers ativos, os principais acessos são:

- **Frontend:** `http://localhost:5173`;
- **Backend:** `http://localhost:8000`;
- **Swagger:** `http://localhost:8000/docs`;
- **Endpoint RPC:** `http://localhost:8000/rpc`;
- **PostgreSQL externo:** `localhost:5433`;
- **PostgreSQL interno no Docker:** `db:5432`.

Para executar os testes:

```
docker compose exec backend pytest -v
```

Exemplo de chamada manual RPC:

```
Invoke-RestMethod `
```

```
-Uri "http://localhost:8000/rpc" `
```

```
-Method POST `
```

```
-ContentType "application/json" `
```

```
-Body '{
```

```
  "jsonrpc": "2.0",
```

```
  "method": "recursos.listar",
```

```
  "params": {},
```

```
  "id": 1
```

```
}'
```

8 SIMULAÇÃO SIMPLIFICADA EM PYTHON

Além do sistema completo, foi criada uma simulação simplificada em Python para fins didáticos. O arquivo `simulacao_leitossync.py` demonstra, de forma reduzida, como clientes leitores e escritores podem chamar procedimentos remotos em um servidor RPC que controla o estado compartilhado dos leitos.

A simulação usa apenas bibliotecas padrão do Python, como `xmlrpc.server`, `xmlrpc.client`, `threading`, `time` e `random`. Ela não substitui o sistema completo, mas ajuda a compreender a lógica central:

- leitores chamam remotamente uma consulta de leitos;
- escritores chamam remotamente uma tentativa de reserva;
- o servidor mantém o recurso compartilhado;
- apenas uma reserva de um mesmo leito é confirmada;
- as demais tentativas recebem conflito.

Para executar:

```
python simulacao_leitossync.py
```

9 LIMITAÇÕES

O projeto foi desenvolvido como uma prova de conceito acadêmica. Suas principais limitações são:

- não há autenticação real com JWT;
- não há controle avançado de permissões por papéis;
- o uso de usuários foi simplificado;
- o ambiente não utiliza múltiplas réplicas reais de backend;
- a distribuição é demonstrada por containers, chamadas RPC e operações concorrentes;
- o projeto não implementa consenso distribuído;
- o PostgreSQL atua como ponto central de consistência;
- o WebSocket propaga eventos, mas não garante consistência por si só.

Essas limitações são aceitáveis para o objetivo do trabalho, que é demonstrar RPC, concorrência e a aplicação do problema dos leitores e escritores.

10 TRABALHOS FUTUROS

Como melhorias futuras, podem ser consideradas:

- autenticação e autorização por papéis;
- uso de logs imutáveis;
- múltiplas réplicas do backend;
- uso de Redis para pub/sub entre instâncias;
- fila de prioridade para reservas emergenciais;
- política de expiração de reservas;
- integração com sistemas hospitalares reais;
- monitoramento com Prometheus e Grafana;
- implantação em ambiente de nuvem.

11 CONSIDERAÇÕES FINAIS

O LeitoSync demonstra a aplicação do problema clássico dos leitores e escritores em um cenário distribuído de gestão hospitalar. O sistema permite múltiplas leituras simultâneas de disponibilidade e controla escritas concorrentes por meio de transações no PostgreSQL e bloqueio pessimista por linha.

A reformulação para JSON-RPC 2.0 tornou a comunicação principal do sistema orientada a chamadas remotas de procedimento. O endpoint `/rpc` recebe as mensagens, identifica o método solicitado e executa a função correspondente no servidor. Com isso, o projeto atende ao requisito de uso de RPC em Sistemas Distribuídos.

Os testes automatizados confirmam que múltiplas consultas podem ocorrer simultaneamente, enquanto múltiplas tentativas concorrentes de reserva resultam em apenas uma escrita confirmada. Dessa forma, o sistema evita reserva duplicada, preserva a consistência dos dados e apresenta uma solução prática e didática para o problema estudado.

12 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724: informação e documentação - trabalhos acadêmicos - apresentação**. Rio de Janeiro: ABNT, 2011.

JSON-RPC WORKING GROUP. **JSON-RPC 2.0 Specification**. 2013.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation: Explicit Locking and SELECT FOR UPDATE**.

PYTHON SOFTWARE FOUNDATION. **Python Documentation: XML-RPC server and client**.